MINING FREQUENT PATTERNS:

Frequent pattern mining in data mining is the process of identifying patterns or associations within a dataset that occur frequently.

This is typically done by analyzing large datasets to find items or sets of items that appear together frequently.

Frequent pattern extraction is an essential mission in data mining that intends to uncover repetitive patterns or item sets in a granted dataset.

It encompasses recognizing collections of components that occur together frequently in a transactional or relational database.

This procedure can offer valuable perceptions into the connections and affiliations among diverse components or features within the data.

FREQUENT ITEM SET MINING METHODS :

1. Frequent item sets, also known as association rules, are a fundamental concept in association rule mining, which is a technique used in data mining to discover relationships between items in a dataset.

   The goal of association rule mining is to identify relationships between items in a dataset that occur frequently together.

2. A frequent item set is a set of items that occur together frequently in a dataset. The frequency of an item set is measured by the support count, which is the number of transactions or records in the dataset that contain the item set.

   For example, if a dataset contains 100 transactions and the item set {milk, bread} appears in 20 of those transactions, the support count for {milk, bread} is 20.

3. Association rule mining algorithms, such as Apriori or FP-Growth, are used to find frequent item sets and generate association rules.
   These algorithms work by iteratively generating candidate item sets and pruning those that do not meet the minimum support threshold.

   Once the frequent item sets are found, association rules can be generated by using the concept of confidence, which is the ratio of the number of transactions that contain the item set and the number of transactions that contain the antecedent (left-hand side) of the rule.

4. Frequent item sets and association rules can be used for a variety of tasks such as market basket analysis, cross-selling and recommendation systems.

   However, it should be noted that association rule mining can generate a large number of rules, many of which may be irrelevant or uninteresting.

   Therefore, it is important to use appropriate measures such as lift and conviction to evaluate the interestingness of the generated rules.

MINING ASSOCIATION RULES

Support

Support is the frequency of A or how frequently an item appears in the dataset. It is defined as the fraction of the transaction T that contains the itemset X. If there are X datasets, then for transactions T, it can be written as:

$$\text{Supp(X)} = \frac{Freq(X)}{T}$$

Confidence

Confidence indicates how often the rule has been found to be true. Or how often the items X and Y occur together in the dataset when the occurrence of X is already given. It is the ratio of the transaction that contains X and Y to the number of records that contain X.

$$\text{Confidence} = \frac{Freq(X,Y)}{Freq(X)}$$

Lift

It is the strength of any rule, which can be defined as below formula:

$$\text{Lift}=\frac{Supp(X,Y)}{Supp(X)\times Supp(Y)}$$

APRIORI ALGORITHM

Apriori algorithm refers to an algorithm that is used in mining frequent products sets and relevant association rules. Generally, the apriori algorithm operates on a database containing a huge number of transactions. For example, the items customers but at a Big Bazar.

Apriori algorithm helps the customers to buy their products with ease and increases the sales performance of the particular store.

WORKING OF APRIORI ALGORITHM

The Apriori algorithm operates on a straightforward premise. When the support value of an item set exceeds a certain threshold, it is considered a frequent item set. Take into account the following steps.

To begin, set the support criterion, meaning that only those things that have more than the support criterion are considered relevant.

- Step 1: Create a list of all the elements that appear in every transaction and create a frequency table.
- Step 2: Set the minimum level of support. Only those elements whose support exceeds or equals the threshold support are significant.
- Step 3: All potential pairings of important elements must be made, bearing in mind that AB and BA are interchangeable.
- Step 4: Tally the number of times each pair appears in a

transaction.

- Step 5: Only those sets of data that meet the criterion of support are significant.

- Step 6: Now, suppose you want to find a set of three things that may be bought together. A rule, known as self-join, is needed to build a three-item set. The item pairings OP, OB, PB, and PM state that two combinations with the same initial letter are sought from these sets.

1. OPB is the result of OP and OB.
2. PBM is the result of PB and PM.

- Step 7: When the threshold criterion is applied again, you'll get the significant item set. **Applications of Apriori Algorithm**

Apriori is used in the following fields:

- Education

Through the use of traits and specializations, data mining of accepted students may be used to extract association rules.

- Medical

Analyzing the patient's database, for example, might be appropriate.

- Forestry

Frequency and intensity of forest fire analysis using forest fire data.

- Autocomplete Tool

Apriori is employed by a number of firms, including Amazon's recommender system and Google's autocomplete tool.

Example : 1

Example of Apriori: Support threshold=50%, Confidence= 60%

TABLE-1

Transaction	List of items
T1	I1,I2,I3
T2	I2,I3,I4

T3	I4,I5
T4	I1,I2,I4
T5	I1,I2,I3,I5
Transaction	**List of items**
T6	I1,I2,I3,I4

Solution:

Support threshold=50% => 0.5*6= 3 => min_sup=3

1. Count Of Each Item

TABLE-2

Item	**Count**
I1	4
I2	5
I3	4
I4	4
I5	2

2. Prune Step: TABLE -2 shows that I5 item does not meet min_sup=3, thus it is deleted, only I1, I2, I3, I4 meet min_sup count.

TABLE-3

Item	**Count**
I1	4
I2	5
I3	4
I4	4

3. Join Step: Form 2-itemset. From **TABLE-1** find out the occurrences of 2-itemset.

TABLE-4

Item	**Count**
I1,I2	4
I1,I3	3
I1,I4	2
I2,I3	4

I2,I4	3
I3,I4	2

4. Prune Step: TABLE -4 shows that item set {I1, I4} and {I3, I4} does not meet min_sup, thus it is deleted.

TABLE-5

Item	**Count**
I1,I2	4
I1,I3	3
I2,I3	4
I2,I4	3

5. Join and Prune Step: Form 3-itemset. From the **TABLE- 1** find out occurrences of 3-itemset. From **TABLE-5**, find out the 2-itemset subsets which support min_sup.

We can see for itemset {I1, I2, I3} subsets, {I1, I2}, {I1, I3}, {I2, I3} are occurring in **TABLE-5** thus {I1, I2, I3} is frequent.
We can see for itemset {I1, I2, I4} subsets, {I1, I2}, {I1, I4}, {I2, I4}, {I1, I4} is not frequent, as it is not occurring in **TABLE-5** thus {I1, I2, I4} is not frequent, hence it is deleted.

TABLE-6

Item
Item
I1,I2,I3
I1,I2,I4
I1,I3,I4
I2,I3,I4

Only {I1, I2, I3} is frequent.

6. Generate Association Rules: From the frequent itemset discovered above the association could be: {I1, I2} => {I3}

Confidence = support {I1, I2, I3} / support {I1, I2} = (3/ 4)* 100 = 75%

{I1, I3} => {I2}

Confidence = support {I1, I2, I3} / support {I1, I3} = (3/ 3)* 100 = 100%

{I2, I3} => {I1}

Confidence = support {I1, I2, I3} / support {I2, I3} = (3/ 4)* 100 = 75%

{I1} => {I2, I3}

Confidence = support {I1, I2, I3} / support {I1} = (3/ 4)* 100 = 75%

{I2} => {I1, I3}

Confidence = support {I1, I2, I3} / support {I2 = (3/ 5)* 100 = 60%

{I3} => {I1, I2}

Confidence = support {I1, I2, I3} / support {I3} = (3/ 4)* 100 = 75%

This shows that all the above association rules are strong if minimum confidence threshold is 60%.

Example : 2

We are given the following data set, and Using the Apriori method, we must locate the frequently occurring itemsets and construct association rules:

Transaction ID	**ItemSet**
T1	a, b
T2	a, b, c
T3	a, b, c, e
T4	b, c, d
T5	a, d

Minimum Support = 2 and minimum confidence = 60%

Solution

1. Create a table that contains the support count(frequency of each item set) of itemsets individually.

Item Set	Support Count
a	4 b 4
c	3
d	2
e	1

After removing an item set with a support count less than minimum support, we get

Item Set	Support Count
a	4
b	4
c	3 d
	2

2. Create a table that contains the support count of itemsets present in the final table of step 1 in pairs

Item Set	Support Count
Item Set	Support Count
a, b	3
a, c	2
a, d	1

b, c	3
b, d	1
c, d	1

After removing an item set with a support count less than minimum support, we get

Item Set	**Support Count**
a, b	3
a, c	2
b, c	3

3. Create a table that contains the support count of itemsets present in the final table of step 1 in triplets.

Item Set	**Support Count**
a, b, c	2
b, c, d	1

After removing an item set with a support count less than minimum support, we get

Item Set	**Support Count**
a, b, c	2

4. Find the association rules for the subsets Create a new table with all possible rules from the occurred combination {a, b, c}.

Rules	**Support**	**Confidence**
{a, b} -> c	2	2/4 = 50%
{b, c} -> a	2	2/3 =66.67%
{a, c} -> b	2	2/2 =100%

a -> {b, c}	2	2/4 =50%
b -> {a, c}	2	2/4 =50%
c -> {a, b}	2	2/3=66.67%

After removing rules with confidence less than minimum confidence, we get

Rules	**Support**	**Confidence**
{b, c} -> a	2	2/3 =66.67%
{a, c} -> b	2	2/2 =100%
c -> {a, b}	2	2/3=66.67%

Rules **Support** **Confidence**

Now we can consider {b, c} -> a, {a, c} -> b, c -> {a, b} as strong association rules for the given problem.

FREQUENT PATTERN GROWTH(Growth) ALGORITHM :

FP Growth in Data Mining

The **FP Growth algorithm** is a popular method for frequent pattern mining in data mining. It works by constructing a **frequent pattern tree (FP-tree)** from the input dataset. The **FP-tree** is a compressed representation of the dataset that captures the frequency and association information of the items in the data.

The algorithm first scans the dataset and maps each transaction to a path in the tree. Items are ordered in each transaction based on their frequency, with the most frequent items appearing first. Once the FP tree is constructed, frequent itemsets can be generated by recursively mining the tree. This is done by starting at the bottom of the tree and working upwards, finding all combinations of itemsets that satisfy the minimum support threshold.

The FP Growth algorithm in data mining has several advantages over other frequent pattern mining algorithms, such as Apriori.

The **Apriori algorithm** is not suitable for handling large datasets because it generates a large number of candidates and requires multiple scans of the database to my frequent items. In comparison, the FP Growth algorithm requires only a single scan of the data and a small amount of memory to construct the FP tree. It can also be **parallelized to improve performance**.

Working on FP Growth Algorithm

The working of the FP Growth algorithm in data mining can be summarized in the following steps:

- **Scan the database:**
  In this step, the algorithm scans the input dataset to determine the frequency of each item. This determines the order in which items are added to the FP tree, with the most frequent items added first.

- **Sort items:**
  In this step, the items in the dataset are sorted in descending order of frequency. The infrequent items that do not meet the minimum support threshold are removed from the dataset. This helps to reduce the dataset's size and improve the algorithm's efficiency.

- **Construct the FP-tree:**
  In this step, the FP-tree is constructed. The FP-tree is a compact data structure that stores the frequent itemsets and their support counts.

- **Generate frequent itemsets:**
  Once the FP-tree has been constructed, frequent itemsets can be generated by recursively mining the tree. Starting at the bottom of the tree, the algorithm finds all combinations of frequent item sets that satisfy the minimum support threshold.

- **Generate association rules:**
  Once all frequent item sets have been generated, the algorithm post-processes the generated frequent item sets to generate association rules, which can be used to identify interesting relationships between the items in the dataset.

FP Tree

The **FP-tree (Frequent Pattern tree)** is a data structure used in the FP Growth algorithm for frequent pattern mining. It represents the frequent itemsets in the input dataset compactly and efficiently. The FP tree consists of the following components:

- **Root Node:**
  The root node of the FP-tree represents an empty set. It has no associated item but a pointer to the first node of each item in the tree.

- **Item Node:**
  Each item node in the FP-tree represents a unique item in the dataset. It stores the item name and the frequency count of the item in the dataset.

- **Header Table:**
  The header table lists all the unique items in the dataset, along with their frequency count. It is used to track each item's location in the FP tree.

- **Child Node:**
  Each child node of an item node represents an item that cooccurs with the item the parent node represents in at least one transaction in the dataset.

- **Node Link:**
  The node-link is a pointer that connects each item in the header table to the first node of that item in the FP-tree. It is used to traverse the conditional pattern base of each item during the mining process.

The FP tree is constructed by scanning the input dataset and inserting each transaction into the tree one at a time.

For each transaction, the items are sorted in descending order of frequency count and then added to the tree in that order.

If an item exists in the tree, its frequency count is incremented, and a new path is created from the existing node.

If an item does not exist in the tree, a new node is created for that item, and a new path is added to the tree. We will understand in detail how FP-tree is constructed in the next section.

Algorithm by Han

Let's understand with an example how the FP Growth algorithm in data mining can be used to mine frequent itemsets. Suppose we have a dataset of transactions as shown below:

Transaction ID	Items
T1	{M, N, O, E, K, Y}
T2	{D, O, E, N, Y, K}
T3	{K, A, M, E}
T4	{M, C, U, Y, K}
T5	{C, O, K, O, E, I}

Let's scan the above database and compute the frequency of each item as shown in the below table.

Item	Frequency
A	1
C	2

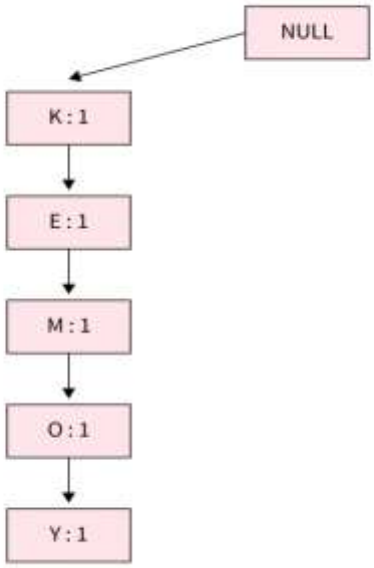
D	1
E	4
I	1
K	5
M	3
N	2
O	3
U	1
Y	3

Let's consider minimum support as 3. After removing all the items below minimum support in the above table, we would remain with these items - {K: 5, E: 4, M : 3, O : 3, Y : 3}. Let's re-order the transaction database based on the items above minimum support. In this step, in each transaction, we will remove infrequent items and re-order them in the descending order of their frequency, as shown in the table below.

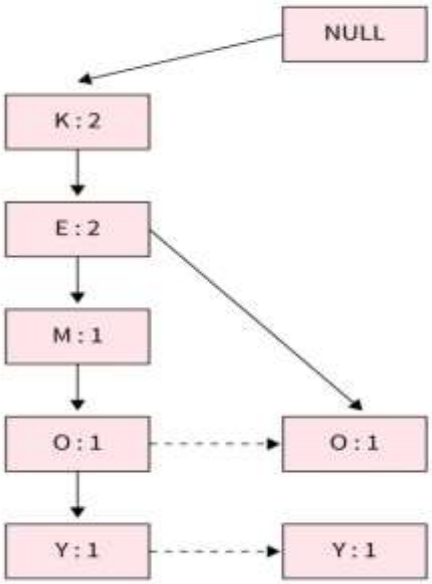
Transaction ID	**Items**	**Ordered Itemset**
T1	{M, N, O, E, K, Y}	{K, E, M, O, Y}
T2	{D, O, E, N, Y, K}	{K, E, O, Y}
T3	{K, A, M, E}	{K, E, M}
T4	{M, C, U, Y, K}	{K, M, Y}
T5	{C, O, K, O, E, I}	{K, E, O}

Now we will use the ordered itemset in each transaction to build the FP tree. Each transaction will be inserted individually to build the FP tree, as shown below -

- **First Transaction {K, E, M, O, Y}:**
  In this transaction, all items are simply linked, and their support count is initialized as 1.

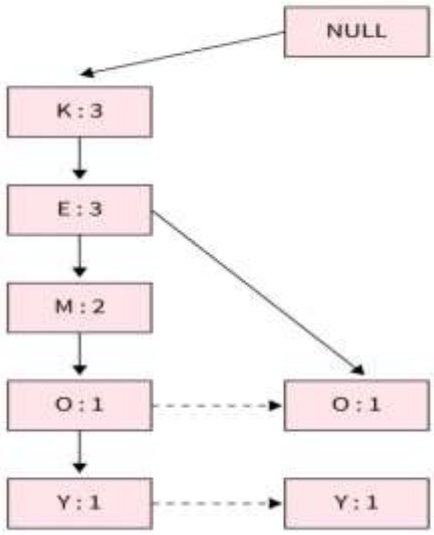


- **Second Transaction {K, E, O, Y}:**
  In this transaction, we will increase the support count of K and E in the tree to 2. As no direct link is available from E to O, we will insert a new path for O and Y and initialize their support count as 1.

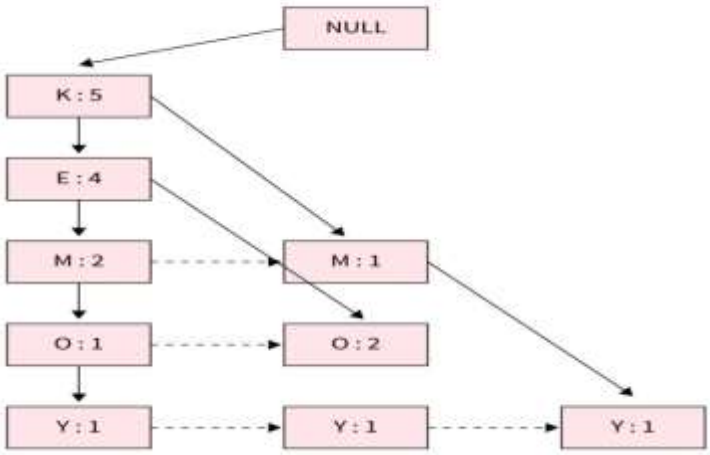


- **Third Transaction {K, E, M}:**
  After inserting this transaction, the tree will look as shown

below. We will increase the support count for K and E to 3 and for M to 2.



- **Fourth Transaction {K, M, Y}** and **Fifth Transaction {K, E, O}:**
  After inserting the last two transactions, the FP-tree will look like as shown below:



Now we will create a **Conditional Pattern Base** for all the items. The conditional pattern base is the path in the tree ending at the given frequent item. For example, for item O, the paths {K, E, M} and {K, E} will result in item O. The conditional pattern base for all items will look like as shown below table:

Item	Conditional Pattern Base
Y	{K, E, M, O : 1}, {K, E, O : 1}, {K, M : 1}

O	{K, E, M : 1}, {K, E : 2}
M	{K, E : 2}, {K : 1}
E	{K : 4}
K	

Now for each item, we will build a conditional frequent pattern tree. It is computed by identifying the set of elements common in all the paths in the conditional pattern base of a given frequent item and computing its support count by summing the support counts of all the paths in the conditional pattern base. The conditional frequent pattern tree will look like this as shown below table:

Item	Conditional Pattern Base	Conditional FP Tree
Y	{K, E, M, O : 1}, {K, E, O : 1}, {K, M : 1}	{K : 3}
O	{K, E, M : 1}, {K, E : 2}	{K, E : 3}
M	{K, E : 2}, {K: 1}	{K : 3}
E	{K: 4}	{K: 4}
K		

From the above conditional FP tree, we will generate the frequent item sets as shown in the below table:

Item	Frequent Patterns
Y	{K, Y - 3}
O	{K, O - 3}, {E, O - 3}, {K, E, O - 3}
M	{K, M - 3}
E	{K, E - 4}

Advantages of FP Growth Algorithm :

The FP Growth algorithm in data mining has several advantages over other frequent itemset mining algorithms, as mentioned below:

- Efficiency:

FP Growth algorithm is faster and more memory-efficient than other frequent itemset mining algorithms such as Apriori, especially on large datasets with high dimensionality. This is because it generates frequent itemsets by constructing the FP-Tree, which compresses the database and requires only two scans.

- Scalability:
  FP Growth algorithm scales well with increasing database

  size and itemset dimensionality, making it suitable for mining frequent itemsets in large datasets.

- Resistant to noise:
  FP Growth algorithm is more resistant to noise in the data than other frequent itemset mining algorithms, as it generates only frequent itemsets and ignores infrequent itemsets that may be caused by noise.

- Parallelization:
  FP Growth algorithm can be easily parallelized, making it suitable for distributed computing environments and allowing it to take advantage of multi-core processors.